Centro Universitário Sagrado Coração — Unisagrado
Curso de Ciências da Computação

Disciplina: Tecnologias e Programação Integrada
Orientador: Prof. Victor Hugo Braguim Canto

LiciteView

Plataforma Inteligente de Avaliação de Oportunidades em Licitações Públicas

Documentação Técnica

Guilherme Jardim

Jhuliani Cristina Santos

Tauani Vitória

Bauru, 2026

Resumo

O presente documento descreve a arquitetura, implementação e decisões técnicas do sistema **LiciteView**, uma plataforma web para avaliação inteligente de oportunidades em licitações públicas brasileiras. O sistema integra o Portal Nacional de Contratações Públicas (PNCP) para busca de editais, persiste documentos PDF diretamente na base de dados relacional, processa-os via pipeline de Recuperação Aumentada por Geração (RAG) em segundo plano, e calcula automaticamente a viabilidade financeira de participação do fornecedor. A solução é composta por um frontend em Angular hospedado no Render, um backend em FastAPI com processamento assíncrono via Celery, e serviços de nuvem para banco de dados (Neon PostgreSQL), armazenamento vetorial (Qdrant Cloud) e comunicação assíncrona (Redis Cloud). O sistema gera resumos estruturados em cinco blocos temáticos, permite consulta em linguagem natural via chat com sugestões dinâmicas de perguntas, mantém histórico de consultas com status no menu lateral, e estima custos operacionais e logísticos utilizando coordenadas geográficas aproximadas e a Tabela TACO da ANTT. Foram realizadas diversas melhorias de usabilidade e resiliência com documentação das decisões técnicas que resultaram em desvios justificados em relação à especificação original.

Palavras-chave: licitações públicas; inteligência artificial; RAG; FastAPI; Angular; Celery; embeddings; PNCP; análise de viabilidade.

Sumário

1 Introdução **5**
1.1 Contexto . . . 5
1.2 Problema . . . 5
1.3 Objetivos . . . 5
1.4 Público-alvo . . . 6

2 Arquitetura do Sistema **6**
2.1 Visão Geral . . . 6
2.2 Frontend . . . 7
2.3 Backend . . . 7
2.4 Fluxo de Processamento do Edital . . . 8
2.5 Fluxo de Consulta ao Edital (RAG) . . . 9
2.6 Fluxo de Análise de Viabilidade . . . 10

3 Tecnologias Utilizadas **12**
3.1 Frontend . . . 12
3.2 Backend . . . 12
3.3 Inteligência Artificial e RAG . . . 13
3.4 Infraestrutura . . . 13

4 Modelagem de Dados **14**
4.1 Diagrama Entidade-Relacionamento . . . 14
4.2 Principais Entidades . . . 14
4.3 Enumerações do Sistema . . . 15

5 Pipeline de Inteligência Artificial **15**
5.1 Extração de Texto — PyMuPDF . . . 15
5.2 Chunking Semântico . . . 16
5.3 Geração de Embeddings — fastembed . . . 16
5.4 Indexação — Qdrant . . . 16
5.5 Geração do Resumo em 5 Blocos . . . 17
5.6 Reranking de Fragmentos — Cohere Rerank . . . 17
5.7 Modelo de Linguagem — Groq . . . 18

6 Referência da API **18**
6.1 Autenticação . . . 18
6.2 Licitações . . . 18
6.3 Editais e Pipeline RAG . . . 19

6.4 Produtos . . . 19
6.5 Análise de Viabilidade . . . 19
6.6 Notificações . . . 20

7 Infraestrutura e Deploy **20**
7.1 Arquitetura de Containers . . . 20
7.2 Dockerfile . . . 21
7.3 Configuração de Ambiente . . . 21
7.4 Nginx e SSL . . . 22
7.5 Como Executar Localmente . . . 22

8 Testes **23**
8.1 Cobertura de Testes . . . 23

9 Decisões Técnicas **24**
9.1 Embeddings: fastembed em vez de OpenAI . . . 24
9.2 RAG: chamadas diretas ao Groq em vez de LangChain . . . 24
9.3 Extração de PDF: PyMuPDF em vez de Docling . . . 24
9.4 Armazenamento de PDFs: banco de dados relacional . . . 24
9.5 Frete: cálculo de distância por CEP com fallback para capitais . . . 25
9.6 Reranking: Cohere Rerank . . . 25
9.7 Proteção de Banco de Dados: Tratamento de Overflow Numérico . . . 25

10 Resultados **26**
10.1 Funcionalidades Implementadas . . . 26
10.2 Métricas de Infraestrutura . . . 27

11 Manual do Usuário **27**
11.1 Cadastro e Login . . . 27
11.1.1 Cadastro . . . 27
11.1.2 Login . . . 27
11.2 Busca de Licitações . . . 28
11.3 Processamento de Edital . . . 28
11.3.1 Processamento automático (via PNCP) . . . 28
11.3.2 Upload manual de PDF . . . 29
11.4 Consultando o Resumo . . . 29
11.5 Análise de Viabilidade . . . 29
11.6 Chat sobre o Edital (IA) . . . 30
11.7 Gerenciamento de Produtos . . . 30
11.7.1 Cadastrar produto . . . 30
11.7.2 Editar ou excluir produto . . . 31

11.8 Notificações 31
11.9 Histórico de Análises 31

12 Próximos Passos (Roadmap) 31

13 Referências 32

A Estrutura de Diretórios do Projeto 32

B Esquema do Banco de Dados 34

1 Introdução

1.1 Contexto

O mercado de compras públicas no Brasil movimenta centenas de bilhões de reais por ano. Segundo dados do Portal Nacional de Contratações Públicas (PNCP), até o final de 2025 haviam sido registradas mais de 1,4 milhão de contratações, totalizando aproximadamente R$ 3 trilhões em valor estimado, com mais de 16 milhões de itens distribuídos em 249 sistemas integrados. Milhares de novas licitações são abertas diariamente em todo o território nacional, representando oportunidades concretas para empresas fornecedoras de produtos e serviços.

1.2 Problema

Para participar de uma licitação pública, uma empresa precisa percorrer um processo extenso e trabalhoso. As etapas envolvem: localizar licitações relevantes entre milhares de publicações diárias; ler editais que frequentemente ultrapassam cem páginas; interpretar requisitos técnicos e jurídicos complexos; estimar custos de produção, impostos e logística; e avaliar se a operação é financeiramente viável antes de decidir participar.

Esse processo é majoritariamente manual e consome tempo significativo dos profissionais responsáveis pela área comercial. Estima-se que um analista possa gastar duas horas ou mais na análise de um único edital. Pequenas e médias empresas (PMEs), que não possuem equipes dedicadas, perdem oportunidades lucrativas por falta de agilidade.

1.3 Objetivos

O objetivo do projeto é desenvolver uma plataforma web que automatize as etapas mais custosas da análise de licitações, entregando:

- **Busca inteligente** de editais no PNCP com filtros por modalidade, UF, esfera e status;
- **Resumo automático** do edital em cinco blocos temáticos gerado por IA;
- **Chat interativo** com o edital por linguagem natural, fundamentado no documento original;
- **Análise de viabilidade** financeira cruzando dados do edital com o produto da empresa;
- **Notificações** assíncronas quando o processamento é concluído.

1.4 Público-alvo

Pequenas e médias empresas fornecedoras de produtos que já participam ou desejam participar do mercado público, mas não possuem ferramentas especializadas de análise. Em especial, vendedores responsáveis pela prospecção e analistas comerciais que precisam avaliar rapidamente quais editais merecem atenção.

2 Arquitetura do Sistema

2.1 Visão Geral

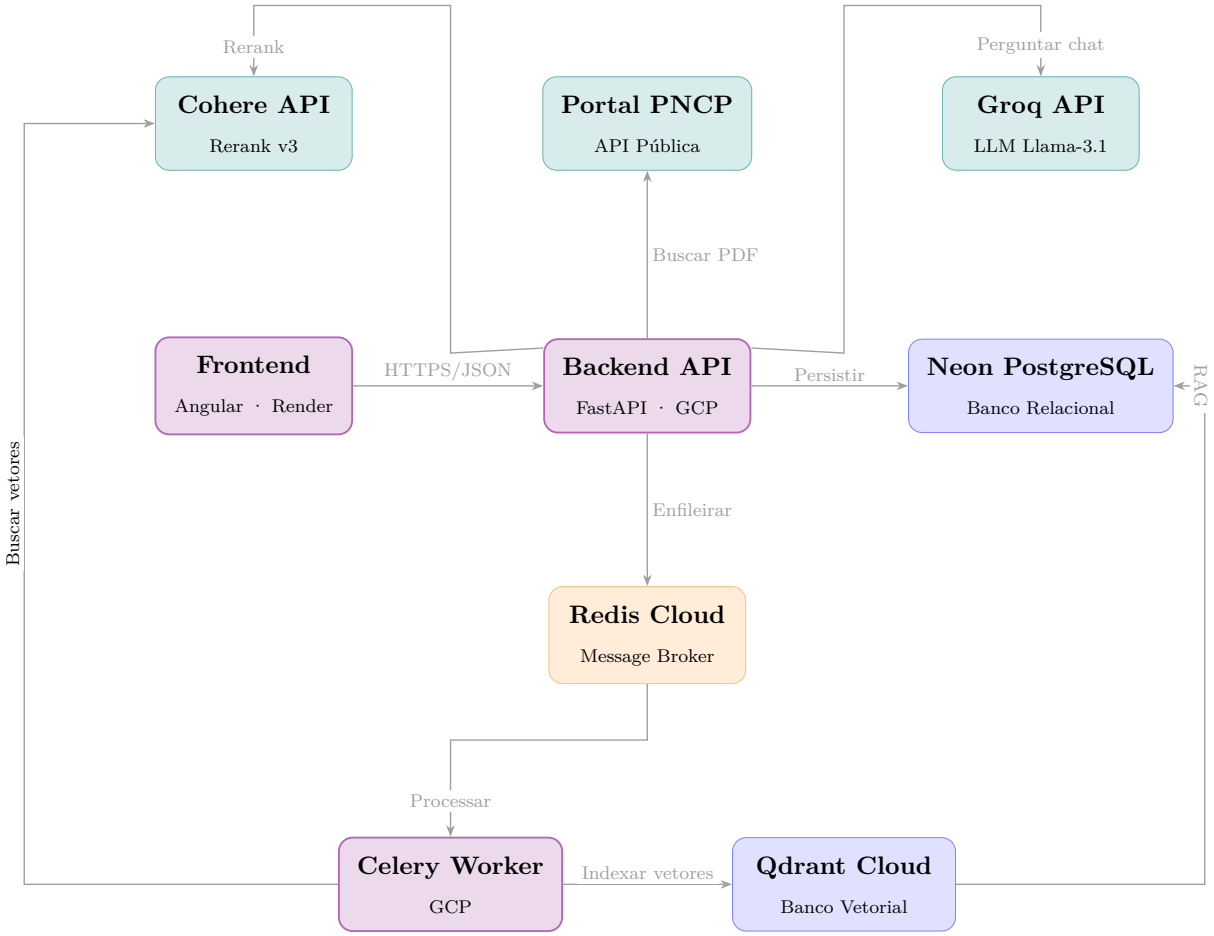


Figura 1: Arquitetura geral do LiciteView

A arquitetura do LiciteView é organizada em cinco camadas com responsabilidades bem definidas:

1. **Camada de apresentação:** frontend Angular responsável pela interface do usuário, hospedado no Render;

2. **Camada de aplicação:** API FastAPI centralizando regras de negócio, autenticação e orquestração, hospedada no Google Cloud Platform;
3. **Camada de serviços assíncronos:** Celery com Redis Cloud para processamento de tarefas pesadas em segundo plano;
4. **Camada de pipeline de IA:** PyMuPDF, fastembed, Qdrant e Groq LLM para processamento documental e geração de respostas;
5. **Camada de dados:** PostgreSQL (Neon) com persistência física de PDFs (coluna `pdf_conteudo`) e Qdrant Cloud.

2.2 Frontend

O frontend em Angular concentra as seguintes funcionalidades:

- Dashboard de radar de oportunidades com filtros por categoria, região e valor;
- Tela de resumo do edital com os cinco blocos temáticos estruturados;
- Módulo de consulta interativa ao edital por linguagem natural;
- Painel de análise de viabilidade com classificação e justificativa;
- Cadastro de produto com definição de margem mínima aceita;
- Sistema de notificações com badge no sino e histórico.

2.3 Backend

A API FastAPI centraliza:

- Autenticação e autorização de usuários via JWT;
- Consulta e ingestão de licitações via API do PNCP;
- Criação de tarefas assíncronas para processamento de editais;
- Endpoints de RAG para consulta ao edital;
- Serviço de análise de viabilidade com cálculo de frete.

2.4 Fluxo de Processamento do Edital

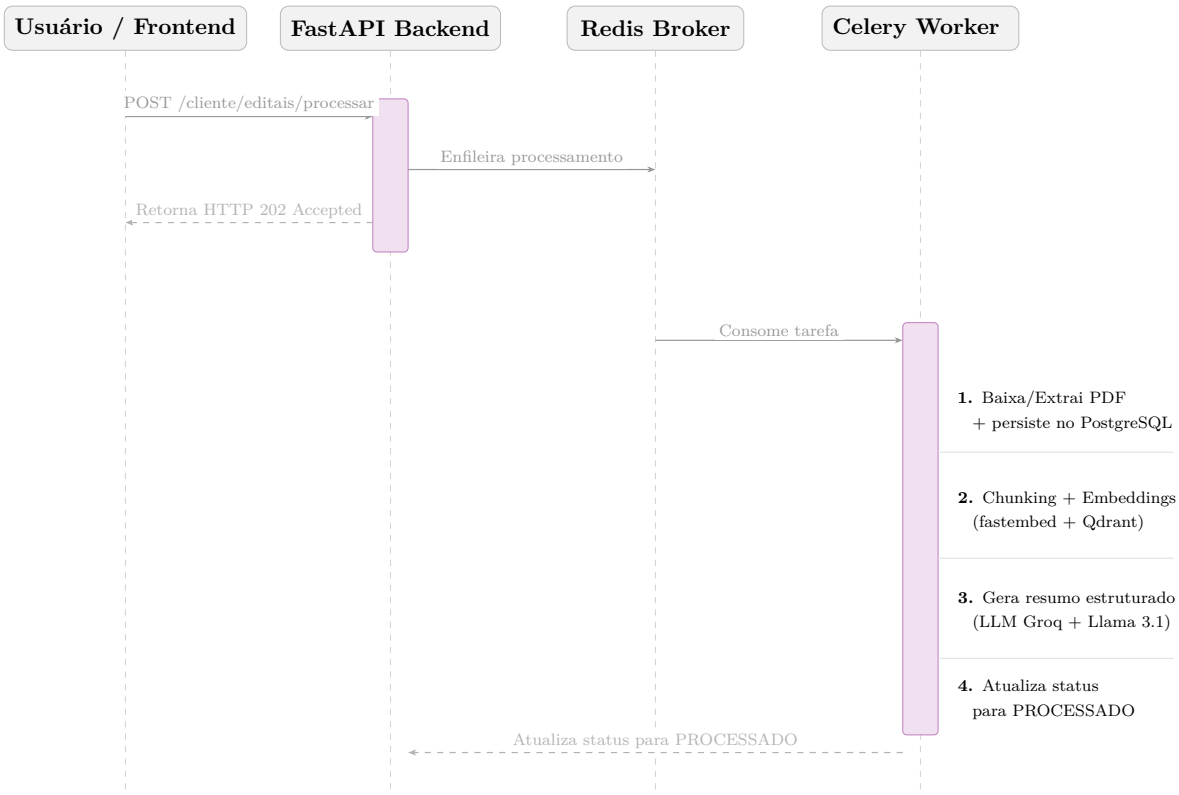


Figura 2: Fluxo de processamento assíncrono do edital

O pipeline de processamento é executado de forma assíncrona pelo Celery e segue as seguintes etapas:

1. O usuário acessa a tela de detalhe de uma licitação;
2. O frontend envia `POST /cliente/editais/processar` com os dados do edital;
3. O backend verifica se já existe resumo em cache no PostgreSQL;
4. Se não existir, cria o registro no banco e enfileira a tarefa no Redis;
5. O worker Celery executa a tarefa assincronamente;
6. O PDF é obtido via API pública do PNCP (ou recebido via upload manual no banco) e salvo na coluna `pdf_conteudo` do PostgreSQL para garantir rastreabilidade e evitar novos downloads;
7. O texto do PDF é extraído com PyMuPDF;
8. O texto é dividido em fragmentos semânticos (450 palavras, sobreposição 75);

9. Os embeddings dos fragmentos são gerados com `all-MiniLM-L6-v2` via fastembed e indexados no Qdrant;

10. Cinco consultas padrão orientadas a blocos temáticos recuperam os trechos mais relevantes;

11. O modelo de linguagem Groq (llama-3.1-8b-instant) organiza as respostas em JSON estruturado;

12. O resumo é salvo no PostgreSQL e o usuário recebe uma notificação no sistema.

2.5 Fluxo de Consulta ao Edital (RAG)

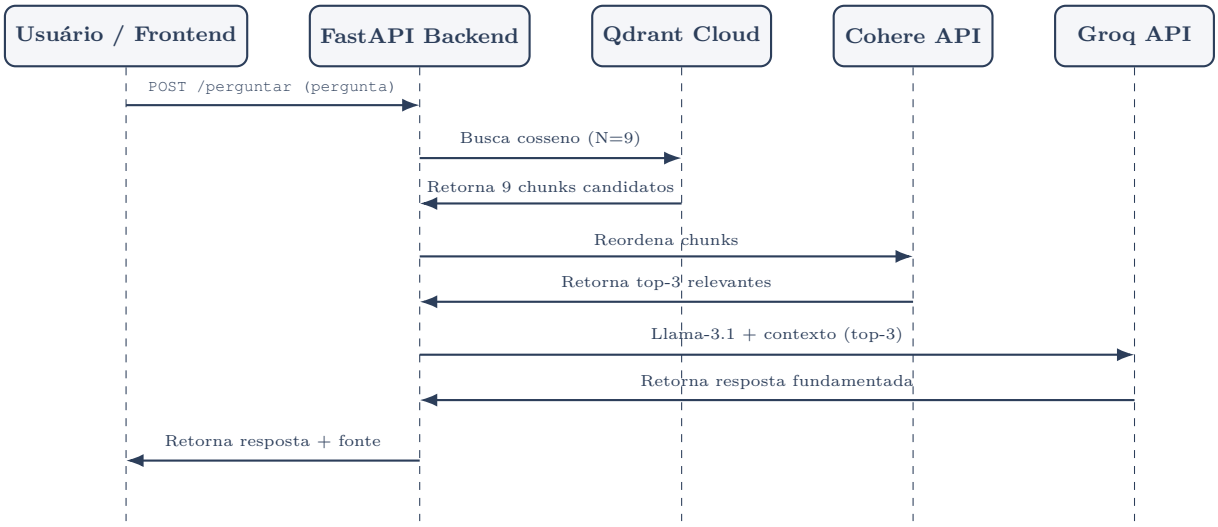


Figura 3: Fluxo de consulta ao edital via RAG com Reranking

Quando o usuário realiza uma pergunta sobre o edital, o fluxo de consulta é orquestrado pelas seguintes etapas:

1. A pergunta é convertida em um vetor de embeddings de 384 dimensões utilizando o modelo `all-MiniLM-L6-v2` via biblioteca **fastembed**;

2. O banco vetorial Qdrant realiza uma busca por similaridade de cosseno e recupera os $N = 9$ fragmentos de texto (chunks) mais semelhantes à pergunta;

3. Os fragmentos recuperados e a pergunta do usuário são enviados ao serviço da **Cohere Rerank** (`rerank-multilingual-v3.0`), que reordena esses 9 candidatos com base em sua relevância semântica real;

4. Apenas os $K = 3$ melhores fragmentos ordenados pelo reranking são selecionados como contexto;

5. O modelo de linguagem Groq (`llama-3.1-8b-instant`) recebe a pergunta e os 3 fragmentos reranqueados, gerando uma resposta fundamentada no edital;

6. A resposta é enviada ao frontend acompanhada do trecho de origem (primeiros 300 caracteres do fragmento mais relevante) e o número da página de onde a informação foi extraída.

2.6 Fluxo de Análise de Viabilidade

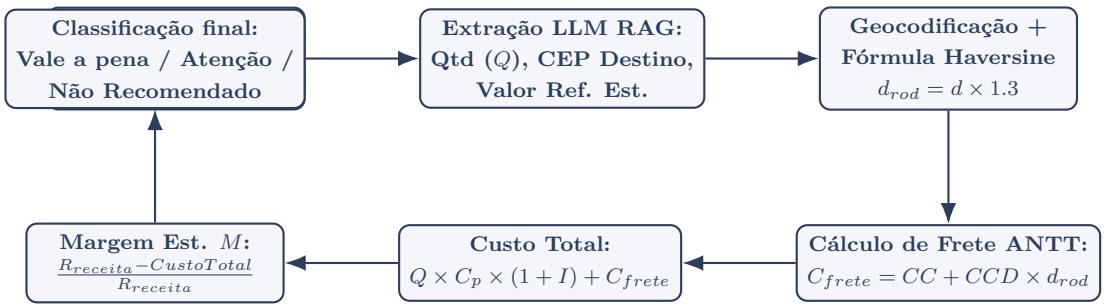


Figura 4: Fluxo de análise de viabilidade financeira

O módulo de viabilidade opera com uma lógica determinística precisa a partir das variáveis extraídas pelo pipeline de inteligência artificial ou inseridas pelo usuário:

1. **Extração de Variáveis com RAG:** O LLM extrai variáveis do edital, como a quantidade demandada (Q), o CEP de entrega (destino), o prazo e o valor unitário estimado. Caso o usuário não preencha o valor estimado do lote, o sistema utiliza busca semântica no Qdrant para extrair automaticamente do edital a quantidade e o valor de referência estimado para o produto associado;

2. **Cálculo de Distância Terrestre (Haversine):** A partir do CEP de origem do produto (cadastrado pelo fornecedor) e do CEP de entrega, são buscadas as coordenadas geográficas (latitude ϕ e longitude λ) através de geocodificação via APIs públicas (BrasilAPI / AwesomeAPI). A distância geodésica em linha reta (d) é calculada pela fórmula de Haversine:

$$\Delta\phi = \phi_2 - \phi_1, \quad \Delta\lambda = \lambda_2 - \lambda_1 \tag{1}$$

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{2}$$

$$d = 2R \cdot \arcsin\left(\sqrt{a}\right) \tag{3}$$

Onde $R = 6371km$ é o raio médio da Terra. Para obter uma aproximação realista do trajeto rodoviário por vias públicas, a distância final é escalada por um fator de

tortuosidade rodoviária ($T = 1.3$):

$$d_{rodoviária} = d \times 1.3 \tag{4}$$

Se a busca de CEP falhar, o sistema adota um fallback seguro baseado em coordenadas predefinidas das capitais estaduais (UFs de origem e destino);

3. **Cálculo do Custo do Frete (ANTT):** O custo estimado do frete (C_{frete}) é calculado com base nos coeficientes da Tabela TACO da ANTT (Portaria SUROC-3/2026) para carga geral de 2 eixos:

$$C_{frete} = CC + CCD \times d_{rodoviária} \tag{5}$$

Onde CC representa o Custo de Carga e Descarga (fixo) e CCD representa o Custo de Deslocamento por quilômetro rodado;

4. **Margem Financeira Estimada:** O custo operacional é obtido pela quantidade de itens (Q), custo unitário do produto (C_p) e a carga tributária (I):

$$CustoOperacional = Q \times C_p \times (1 + I) \tag{6}$$

O custo total é a soma do custo operacional com o frete estimado:

$$CustoTotal = CustoOperacional + C_{frete} \tag{7}$$

Comparando a receita de referência esperada ($R_{receita}$) com o custo total, a margem financeira estimada (M) é calculada por:

$$M = \frac{R_{receita} - CustoTotal}{R_{receita}} \tag{8}$$

Esta margem M é limitada por um limitador inferior e superior no banco de dados, $M \in [-9.9999, 9.9999]$, prevenindo falhas de overflow no PostgreSQL;

5. **Classificação de Viabilidade:** A oportunidade é classificada em três categorias:

   - **Vale a pena** (Verde): Margem estimada superior ou igual à margem mínima definida pelo fornecedor ($M \geq M_{mínima}$);
   - **Atenção** (Laranja): O edital gera lucro, mas a margem está abaixo da mínima esperada ($0 \leq M < M_{mínima}$);
   - **Não recomendado** (Vermelho): O custo total estimado supera a receita de referência, gerando prejuízo operacional ($M < 0$).

3 Tecnologias Utilizadas

3.1 Frontend

Tabela 1: Tecnologias do frontend

Tecnologia	Versão	Finalidade
Angular	17	Framework SPA com componentes standalone e signals
TypeScript	5.4	Tipagem estática
PrimeNG	17	Biblioteca de componentes UI
RxJS	7.8	Gerenciamento de streams assíncronos
Jasmine	—	Testes unitários dos serviços

O Angular foi escolhido pela adequação ao domínio da aplicação, que exige formulários reativos complexos, tabelas com filtros e dashboards corporativos. A versão 17 introduz o sistema de *signals* que simplifica o gerenciamento de estado reativo.

3.2 Backend

Tabela 2: Tecnologias do backend

Tecnologia	Versão	Finalidade
Python	3.10	Linguagem principal
FastAPI	0.136	Framework web assíncrono com suporte nativo a Pydantic
SQLAlchemy	2.0	ORM para PostgreSQL
Alembic	1.18	Migrations de banco de dados
Celery	5.6	Processamento assíncrono de tarefas
Redis	7.4	Broker de mensagens para Celery
Pydantic	2.13	Validação e serialização de dados
python-jose	3.5	Geração e validação de JWT
bcrypt	5.0	Hash de senhas
httpx	0.28	Cliente HTTP assíncrono
slowapi	0.1.9	Rate limiting

3.3 Inteligência Artificial e RAG

Tabela 3: Tecnologias de IA e RAG

Tecnologia	Versão	Finalidade
fastembed	0.7	Geração de embeddings com ONNX (sem PyTorch)
all-MiniLM-L6-v2	—	Modelo de embeddings (384 dimensões)
Qdrant Client	1.17	Interface com o banco vetorial
Groq SDK	1.2	Cliente para o LLM llama-3.1-8b-instant
PyMuPDF	1.24	Extração de texto estruturado de PDFs
pypdf	6.11	Fallback para extração de PDFs

3.4 Infraestrutura

Tabela 4: Infraestrutura e serviços em nuvem

Serviço	Plano	Uso
Google Cloud Platform	e2-medium (2 vCPU, 4 GB)	Hospedagem da API e worker
Render	Free tier	Hospedagem do frontend Angular
Neon PostgreSQL	Free tier	Banco de dados relacional
Qdrant Cloud	Free tier	Banco de dados vetorial
Redis Cloud	Free tier	Broker de mensagens Celery
Groq API	Free tier	LLM llama-3.1-8b-instant
DuckDNS	Gratuito	DNS dinâmico para domínio
Let's Encrypt	Gratuito	Certificado SSL/TLS

4 Modelagem de Dados

4.1 Diagrama Entidade-Relacionamento

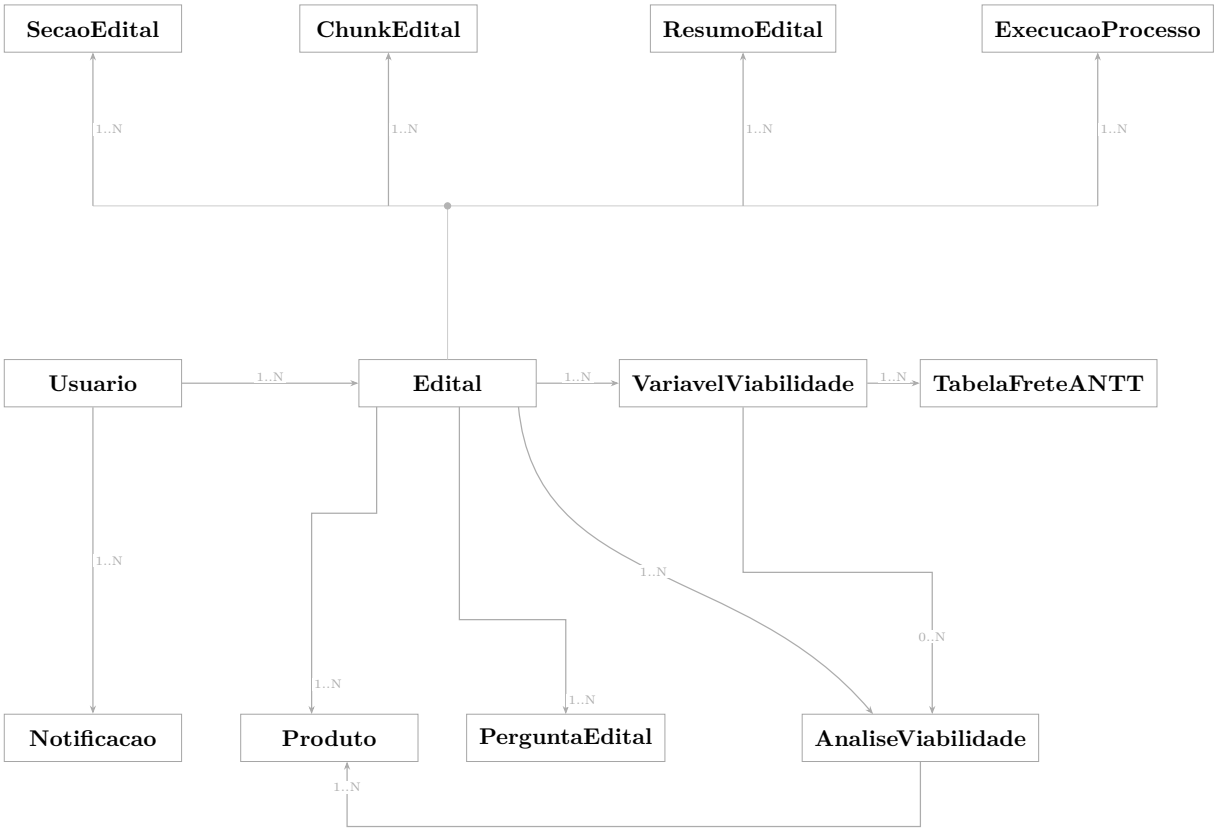


Figura 5: Diagrama Entidade-Relacionamento do LiciteView

4.2 Principais Entidades

Tabela 5: Principais entidades do sistema

Entidade	Chave Primária	Descrição
Usuario	UUID	Fornecedor cadastrado. Armazena dados de autenticação, razão social, CNPJ e CEP
Produto	UUID	Item ofertado pelo usuário. Define custo, margem mínima, impostos e localização de origem
Edital	UUID	Documento oficial da licitação, obtido via PNCP. Armazena metadados e status de processamento

(continuação)

Entidade	Chave Primária	Descrição
ChunkEdital	UUID	Fragmento de texto do edital após chunking. Base para indexação vetorial no Qdrant
ResumoEdital	UUID	Resumo em JSON gerado pelo pipeline RAG. Armazena os 5 blocos temáticos versionados
VariavelViabilidade	UUID	Variável tipada extraída do resumo via LLM para uso nos cálculos
AnaliseViabilidade	UUID	Resultado do cálculo de viabilidade financeira para um par edital/produto
PerguntaEdital	UUID	Pergunta feita pelo usuário sobre o edital. Inclui resposta e trecho de origem
TabelaFreteANTT	UUID	Tabela TACO da ANTT (SUROC-3-2026) com coeficientes CC e CCD por tipo de carga
Notificacao	UUID	Notificação de conclusão ou erro de processamento enviada ao usuário
ExecucaoProcesso	UUID	Log de execução de cada etapa do pipeline (INGESTAO_PDF, EMBEDDING, RESUMO, etc.)

4.3 Enumerações do Sistema

Tabela 6: Enumerações utilizadas

Enumeração	Valores
StatusProcessamento	PENDENTE, PROCESSANDO, PROCESSADO, ERRO
ClassificacaoViabilidade	VALE_A_PENA, ATENCAO, NAO_RECOMENDADO
StatusPergunta	PENDENTE, RESPONDIDA, ERRO
PerfilUsuario	ADMIN, FORNECEDOR
EmbeddingStatus	PENDENTE, GERADO, ERRO
TipoProcesso	INGESTAO_PDF, SEGMENTACAO, EMBEDDING, RESUMO, ANA

5 Pipeline de Inteligência Artificial

5.1 Extração de Texto — PyMuPDF

O sistema utiliza **PyMuPDF** (`fitz`) para extração de texto estruturado dos editais em PDF. A biblioteca analisa os blocos de texto de cada página e detecta hierarquia de

seções com base no tamanho da fonte:

```
for page in doc:
    blocos = page.get_text("dict").get("blocks", [])
    for bloco in blocos:
        for linha in bloco.get("lines", []):
            tamanho_max = max(s.get("size", 10) for s in spans)
            if tamanho_max > 14:
                paragrafos.append(f"# {texto_bloco}")  # titulo
            elif tamanho_max > 12:
                paragrafos.append(f"## {texto_bloco}")  # subtitulo
            else:
                paragrafos.append(texto_bloco)
```

Listing 1: Extração estruturada com PyMuPDF

O sistema detecta automaticamente se o arquivo baixado é um ZIP (magic bytes `PK\x03\x04`) e extrai o PDF interno, pois alguns editais do PNCP são disponibilizados nesse formato.

5.2 Chunking Semântico

O texto extraído é dividido em fragmentos de **350 palavras** com sobreposição de **50 palavras**, garantindo que informações em fronteiras de chunk não sejam perdidas.

5.3 Geração de Embeddings — fastembed

Os fragmentos são vetorizados com o modelo `sentence-transformers/all-MiniLM-L6-v2` via biblioteca **fastembed**, que utiliza ONNX Runtime em CPU:

- Dimensão vetorial: 384
- Modelo: all-MiniLM-L6-v2 (multilíngue)
- Runtime: ONNX (sem dependência de PyTorch)
- Uso de RAM: 200 MB

O modelo é pré-carregado durante o *build* do Docker, eliminando downloads em runtime.

5.4 Indexação — Qdrant

Os vetores são armazenados no Qdrant Cloud com uma coleção por edital, nomeada `edital_{uuid_sem_traços}`. A similaridade utilizada é **Cosseno**.

5.5 Geração do Resumo em 5 Blocos

Para cada bloco temático, o sistema realiza uma consulta semântica ao Qdrant e envia o contexto recuperado ao LLM com um schema JSON esperado:

Tabela 7: Blocos temáticos do resumo do edital

Bloco	Tema	Campos principais
bloco1_visao_geral	Visão geral	objeto, órgão, modalidade, critério
bloco2_prazos	Prazos e execução	prazo_entrega, local, vigência
bloco3_documentacao	Documentação	habilitação, regularidade fiscal
bloco4_requisitos_tecnicos	Requisitos técnicos	especificações, certificações, amostra
bloco5_riscos	Riscos e penalidades	multas, sanções, cláusulas críticas

Entre cada chamada ao LLM é inserido um delay de 1.2 segundos para respeitar o limite de *tokens per minute* do plano gratuito do Groq.

5.6 Reranking de Fragmentos — Cohere Rerank

Após a busca por similaridade de cosseno realizada no Qdrant, o sistema utiliza uma etapa adicional de reordenação de fragmentos (reranking) no fluxo de chat. A busca inicial em bancos vetoriais, baseada puramente em similaridade de distância vetorial, muitas vezes recupera trechos que possuem termos semanticamente próximos, mas que não respondem de fato à intenção do usuário.

Para contornar essa limitação e maximizar a precisão das respostas fornecidas pela LLM:

- O backend recupera um número expandido de candidatos ($N = 9$ chunks);
- Esses chunks e a pergunta original são submetidos à API da **Cohere Rerank** através do modelo `rerank-multilingual-v3.0`;
- O modelo calcula uma pontuação de relevância direta entre cada trecho de texto e a pergunta, ordenando-os de forma decrescente;
- Apenas os $K = 3$ fragmentos com maior pontuação são selecionados para compor o contexto final do prompt;
- Se a chave de API da Cohere não estiver configurada ou ocorrer alguma falha de comunicação, o sistema realiza um *fallback* automático e transparente para a ordem por similaridade de cosseno obtida do Qdrant.

5.7 Modelo de Linguagem — Groq

- **Modelo:** `llama-3.1-8b-instant`
- **Temperatura:** 0 (resposta determinística, reduz alucinações)
- **Formato de resposta:** `json_object` (para o resumo estruturado) e texto corrido (para o chat)
- **top_k:** 3 fragmentos de contexto final por chamada
- **Contexto máximo por chamada:** 3.500 caracteres

6 Referência da API

A API segue o padrão REST e retorna respostas em JSON. Todos os endpoints protegidos exigem o cabeçalho `Authorization: Bearer {token}`.

6.1 Autenticação

Tabela 8: Endpoints de autenticação

Método	Endpoint	Body	Resposta
POST	/auth/register	{nome, email, senha, razao_social, cnpj, cep}	UsuarioResponse (201)
POST	/auth/login	{email, senha}	{access_token} (200)
GET	/auth/me	—	UsuarioResponse (200)

6.2 Licitações

Tabela 9: Endpoints de licitações

Método	Endpoint	Parâmetros	Resposta
GET	/cliente/licitacoes/buscar	q, status, ufs, modalidades, pagina	Resultados PNCP (200)

6.3 Editais e Pipeline RAG

Tabela 10: Endpoints de editais

Método	Endpoint	Body/Params	Resposta
POST	/cliente/editais/processar	{id_pncp, orgao, objeto, url_pdf}	{edital_id, status} (202)
GET	/cliente/editais/{id}/status	—	EditalStatusResponse (200)
POST	/cliente/editais/{id}/perguntar	{pergunta}	{resposta, trecho_fonte} (200)

6.4 Produtos

Tabela 11: Endpoints de produtos (requer autenticação)

Método	Endpoint	Body/Params	Resposta
GET	/cliente/produtos	—	list[ProdutoResponse]
POST	/cliente/produtos	ProdutoCreate	ProdutoResponse (201)
PUT	/cliente/produtos/{id}	ProdutoUpdate	ProdutoResponse
DELETE	/cliente/produtos/{id}	—	204 No Content

6.5 Análise de Viabilidade

Tabela 12: Endpoints de análise de viabilidade

Método	Endpoint	Body/Params	Resposta
POST	/cliente/analise	{edital_id, produto_id}	AnaliseViabilidadeRespo
DELETE	/cliente/analise/{edital}/{produto}	—	204 No Content

6.6 Notificações

Tabela 13: Endpoints de notificações

Método	Endpoint	Params	Resposta
GET	/cliente/notificacoes	—	list[NotificacaoRespo
PATCH	/cliente/notificacoes/{id}/lida	—	204 No Content
PATCH	/cliente/notificacoes/marcar-todas-lidas	—	204 No Content
GET	/health	—	{status: "ok"}

7 Infraestrutura e Deploy

7.1 Arquitetura de Containers

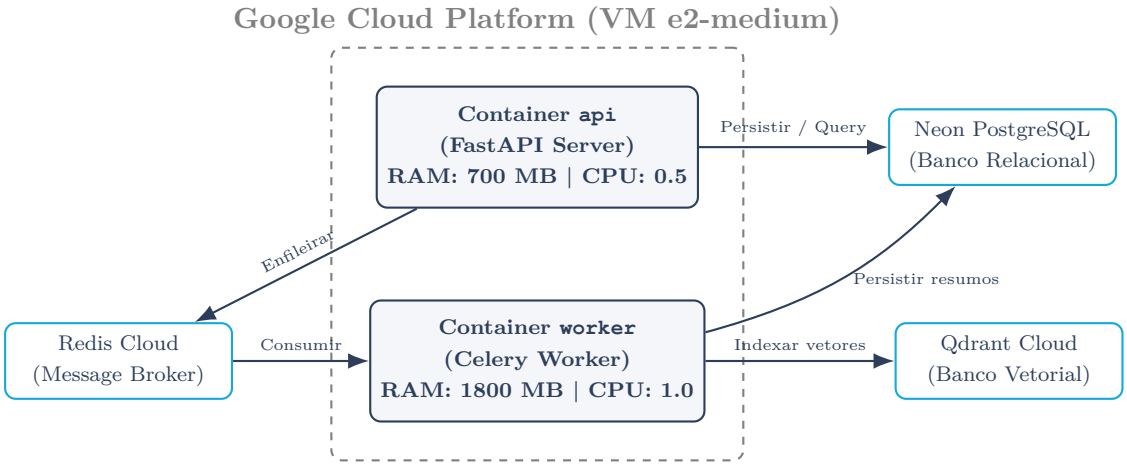


Figura 6: Arquitetura de containers Docker no GCP

O sistema é implantado com dois containers Docker separados no Google Cloud Platform:

- **Container `api`:** executa o servidor FastAPI. Limitado a 700 MB de RAM e 0,5 vCPU. Não executa operações de ML.
- **Container `worker`:** executa o worker Celery. Limitado a 1.800 MB de RAM e 1,0 vCPU. Responsável pelo processamento de PDFs e geração de embeddings.

A separação é justificada pelo isolamento de falhas: se o worker travar por processamento pesado, a API continua respondendo normalmente.

7.2 Dockerfile

O Dockerfile foi simplificado após a migração de Docling para PyMuPDF, removendo dependências de sistema desnecessárias:

```
FROM python:3.10-slim

# PyMuPDF usa wheel binario proprio - nao precisa de libmupdf
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Pre-carrega modelo fastembed na imagem (evita download no restart)
RUN python -c "from fastembed import TextEmbedding; \
               TextEmbedding('sentence-transformers/all-MiniLM-L6-v2')"

COPY . .
RUN rm -rf venv/ __pycache__/ .git/ static/

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--
    workers", "1"]
```

Listing 2: Dockerfile de produção

7.3 Configuração de Ambiente

As seguintes variáveis de ambiente são necessárias para a execução do sistema:

Tabela 14: Variáveis de ambiente

Variável	Descrição
DATABASE_URL	URL de conexão PostgreSQL (Neon)
SECRET_KEY	Chave secreta para assinatura JWT
ALGORITHM	Algoritmo JWT (HS256)
ACCESS_TOKEN_EXPIRE_MINUTES	Tempo de expiração do token
REDIS_HOST	Host:porta do Redis Cloud
REDIS_PASSWORD	Senha do Redis Cloud
QDRANT_HOST	URL do cluster Qdrant Cloud
QDRANT_API_KEY	Chave de API do Qdrant Cloud
GROQ_API_KEY	Chave de API do Groq
ALLOWED_ORIGINS	Origens permitidas pelo CORS
START_CELERY_WORKER	true (dev local) / false (Docker)

7.4 Nginx e SSL

O Nginx atua como proxy reverso na VM, encaminhando requisições HTTPS para o container da API:

- Certificado SSL emitido pelo Let's Encrypt via Certbot;
- Domínio: `licite.duckdns.org` (DuckDNS, IP dinâmico atualizado a cada 5 minutos via cron);
- Redirecionamento automático de HTTP para HTTPS;
- Timeout aumentado para 120s para suportar o pipeline RAG.

7.5 Como Executar Localmente

```
# 1. Configurar variáveis de ambiente
cd licitacoes-ws
cp .env.example .env
# editar .env com as credenciais

# 2. Criar ambiente virtual e instalar dependências
python -m venv venv
venv\Scripts\activate  # Windows
pip install -r requirements.txt

# 3. Aplicar migrations
python -m alembic upgrade head
```

```
13
14 # 4. Iniciar backend (sobe Celery worker automaticamente)
15 uvicorn main:app --reload
16
17 # 5. Iniciar frontend (em outro terminal)
18 cd liciteview-angular
19 npm install
20 ng serve
```

Listing 3: Inicialização do ambiente de desenvolvimento

8 Testes

O projeto adota testes unitários nos serviços Angular utilizando o framework **Jasmine** com `HttpClientTestingModule` para simular chamadas HTTP sem atingir o servidor real.

8.1 Cobertura de Testes

Tabela 15: Arquivos de teste e casos cobertos

Arquivo	**Casos testados**
auth.service.spec.ts	Login chama POST correto; token salvo no localStorage; userDisplay retorna iniciais; logout remove token; isAuthenticated com e sem token
licitacao.service.spec.ts	Search chama URL correta; mapeamento de status; filtro UF; status 'todos' sem parâmetro; conversão PncpItem→Licitacao; total do servidor; cache após search
produto.service.spec.ts	GET na inicialização; POST com body correto e signal atualizado; DELETE com signal reduzido; PUT com item atualizado
analise.service.spec.ts	POST com edital_id e produto_id; classificação ATENCAO→amber; signal armazenado; clear remove signal; custos mapeados corretamente
edital.service.spec.ts	POST processar com body correto; GET status no endpoint correto; poll emite até terminal; poll para em PROCESSADO (inclusive); perguntar chama POST correto

9 Decisões Técnicas

Esta seção registra os principais desvios em relação à especificação original do projeto e as justificativas correspondentes.

9.1 Embeddings: fastembed em vez de OpenAI

A especificação previa o uso do modelo `text-embedding-3-small` da OpenAI. Foi implementado o modelo `all-MiniLM-L6-v2` via fastembed por três razões principais:

1. **Custo zero:** elimina custo por token para o volume de chunks processados;
2. **Independência de API:** funciona offline e não depende de quotas externas;
3. **Eficiência operacional:** fastembed com ONNX ocupa 50 MB vs. 1,5 GB do PyTorch.

9.2 RAG: chamadas diretas ao Groq em vez de LangChain

O LangChain foi avaliado mas preterido pela sua camada de abstração que dificulta o debug. O pipeline do sistema é linear e previsível, não justificando o uso de chains complexas.

9.3 Extração de PDF: PyMuPDF em vez de Docling

O Docling foi implementado inicialmente mas removido porque trazia `torch` (1,5 GB) via `docling-ibm-models`, tornando a imagem Docker com 6 GB e travando a VM e2-medium (4 GB RAM) durante o build. PyMuPDF resolve o mesmo problema com uma biblioteca C de 20 MB sem dependências de ML.

9.4 Armazenamento de PDFs: banco de dados relacional

Os PDFs dos editais (tanto obtidos do PNCP quanto carregados manualmente pelo usuário) são armazenados na coluna `pdf_conteudo` (tipo `LargeBinary`) no próprio banco relacional PostgreSQL. Justificativa: essa abordagem garante a persistência física dos documentos para auditorias de viabilidade, histórico de consultas e reprocessamento offline, eliminando a dependência em tempo real da API do PNCP para downloads repetitivos, sem incorrer na complexidade de provisionar o Google Cloud Storage para o MVP.

9.5 Frete: cálculo de distância por CEP com fallback para capitais

Para obter estimativas precisas de frete sem a necessidade de chaves comerciais e pagas de APIs como Google Maps, o sistema calcula a distância geodésica terrestre de forma dinâmica. A partir do CEP do produto cadastrado (origem) e do CEP de entrega extraído do edital (destino), a aplicação consulta APIs públicas (BrasilAPI com fallback rápido para a AwesomeAPI) para obter coordenadas reais de latitude e longitude. A distância em quilômetros é calculada via fórmula de Haversine com a aplicação de um fator de tortuosidade rodoviária de 1.3 para aproximar o trajeto terrestre real. Caso a consulta dos CEPs falhe ou esteja offline, o sistema realiza um fallback gracioso para a tabela de distâncias aproximadas entre as capitais dos respectivos estados.

9.6 Reranking: Cohere Rerank

O reranking com o modelo `rerank-multilingual-v3.0` da Cohere foi integrado com sucesso no pipeline de chat (RAG). Em cada pergunta feita pelo usuário, o sistema busca $top_k \times 3$ chunks candidatos do banco vetorial Qdrant, e a API da Cohere reordena esses blocos conforme a relevância real para a pergunta. Apenas os top_k melhores trechos reranqueados são passados ao contexto da LLM, aumentando expressivamente a precisão da resposta. Em caso de ausência da API Key ou erros na rede, o sistema faz um fallback silencioso e utiliza os trechos originais do Qdrant por similaridade de cosseno.

9.7 Proteção de Banco de Dados: Tratamento de Overflow Numérico

Durante testes de estresse em cenários com prejuízo severo (ex: custo logístico de frete extremamente superior à receita total estimada), observou-se que a margem financeira calculada assumia valores negativos que excediam a precisão definida na coluna `margem_estimada` (`NUMERIC(5, 4)`). Essa inconsistência de dados gerava exceções de estouro numérico (`NumericValueOutOfRange`) no PostgreSQL, resultando em quebras do servidor (erro HTTP 500). Para garantir a estabilidade do sistema relacional, o backend foi adaptado para aplicar uma restrição estrita (*clamping*) no valor final da margem entre -9.9999 e 9.9999 antes de persistir, mantendo a estimativa percentual irrestrita apenas nas descrições de texto da justificativa geradas para o usuário.

Tabela 16: Resumo dos desvios da especificação

Componente	Especificado	Implementado	Impacto
Embeddings	OpenAI 3-small	fastembed MiniLM	Menor custo
Orquestração	LangChain	Direto Groq	Mais simples
Reranking	Cohere v3	Cohere Rerank	Chat mais preciso
Storage PDF	Google Cloud	Banco PostgreSQL	Salvo em `pdf_conteudo`
Extração PDF	Docling	PyMuPDF	Imagem menor
Distância frete	CEP → coordenadas	Geocodificação BrasilAPI + Haversine (1.3)	Alta precisão

10 Resultados

10.1 Funcionalidades Implementadas

Tabela 17: Status das funcionalidades

Funcionalidade	Status	Observação
Autenticação (registro/login/JWT)	Implementado	—
Busca de licitações (PNCP)	Implementado	Paginação, 12 filtros
Cadastro de produto	Implementado	CRUD completo
Processamento assíncrono (Celery)	Implementado	Redis + polling
Busca automática do PDF (PNCP API)	Implementado	—
Extração e chunking do PDF	Implementado	PyMuPDF + pypdf fallback
Geração de embeddings	Implementado	fastembed MiniLM-L6-v2
Indexação vetorial (Qdrant)	Implementado	—
Resumo em 5 blocos (RAG + LLM)	Implementado	Groq llama-3.1-8b
Chat sobre o edital	Implementado	Validação de status de processamento
Sugestão de perguntas no chat	Implementado	Atalhos rápidos no chat
Análise de viabilidade	Implementado	ANTT + Haversine + Valor ref. manua
Histórico de consultas	Implementado	Menu lateral com status de progresso
Notificações no sino	Implementado	Polling 30s
Deploy em produção (GCP)	Implementado	Docker + Nginx + SSL
Reranking (Cohere)	Implementado	Modelo rerank-multilingual-v3.0 integra

10.2 Métricas de Infraestrutura

Tabela 18: Métricas do ambiente de produção

Métrica	Valor
Tempo de build Docker (com fastembed)	~15 minutos
Tamanho da imagem Docker	~500 MB
RAM em uso — container API	~200 MB
RAM em uso — container Worker (idle)	~300 MB
Tempo de processamento de edital (30p.)	60–120 segundos
Tempo de resposta ao chat	3–8 segundos
Requests de polling por edital	~15 (intervalo 8s)

11 Manual do Usuário

Acesso ao sistema

URL de produção: `https://liciteview.onrender.com`

Email: `admin@empresa.com.br`

Senha: `12345678`

11.1 Cadastro e Login

11.1.1 Cadastro

1. Acesse a URL do sistema e clique em **"Criar conta"**.
2. Preencha os campos obrigatórios: nome completo, e-mail, senha (mín. 8 caracteres), CNPJ, CEP, cidade e estado (UF).
3. Clique em **"Cadastrar"**. Você será redirecionado automaticamente para a tela de pesquisa.

11.1.2 Login

1. Informe seu **e-mail** e **senha**.
2. Clique em **"Entrar"**.
3. Em caso de credenciais inválidas, o sistema exibe uma mensagem de erro.

11.2 Busca de Licitações

A tela principal após o login é a **Pesquisa de Licitações**.

1. No campo **"Buscar por palavra-chave"**, digite o produto ou serviço desejado (ex: *luvas descartáveis*).
2. Utilize os filtros disponíveis:
   - **Estado (UF):** selecione um estado ou "Todos"
   - **Status:** ex: "Recebendo proposta", "Homologada"
   - **Modalidade:** ex: "Pregão Eletrônico", "Dispensa"
   - **Esfera:** Federal, Estadual ou Municipal
3. Clique em **"Pesquisar"**.
4. Os resultados aparecem em tabela com: objeto, órgão, UF, valor estimado e status.
5. Clique no **ícone de detalhes** na linha desejada para abrir o edital.

Dica

Use termos específicos do objeto para obter resultados mais precisos. O sistema busca em tempo real na base do PNCP.

11.3 Processamento de Edital

Ao abrir um edital, você verá três abas: **Resumo**, **Viabilidade** e **Chat**.

11.3.1 Processamento automático (via PNCP)

1. O sistema detecta automaticamente se o edital ainda não foi processado.
2. Clique em **"Processar Edital"** — o sistema baixará o PDF e iniciará a análise.
3. Uma **barra de progresso** exibirá as etapas em andamento:
   - Extração de texto do PDF
   - Geração de embeddings e indexação vetorial
   - Criação do resumo por IA
   - Extração de variáveis-chave
4. O processo leva entre **1 e 3 minutos**.
5. Ao concluir, você receberá uma **notificação** (sino no topo) e as abas serão desbloqueadas.

11.3.2 Upload manual de PDF

1. Clique em **"Upload Manual de PDF"**.
2. Selecione o arquivo `.pdf` no seu computador.
3. O processamento segue os mesmos passos acima.

11.4 Consultando o Resumo

Após o processamento, a aba **Resumo** exibe:

- **5 blocos temáticos** gerados por IA: Objeto, Requisitos, Prazos, Condições Financeiras e Observações Importantes.
- **Variáveis extraídas:** prazo de entrega, valor estimado por item, localidade de entrega e estado de destino.

Observação

Essas informações são usadas automaticamente no cálculo de viabilidade. Você pode editá-las manualmente caso necessário.

11.5 Análise de Viabilidade

Na aba **Viabilidade**:

1. Selecione um **produto do seu catálogo** no dropdown. Se o catálogo estiver vazio, cadastre um produto primeiro (seção 11.7).
2. Clique em **"Calcular Viabilidade"**.
3. O sistema exibirá:
    - **Semáforo de viabilidade:** verde (vale a pena), amarelo (atenção) ou vermelho (não recomendado)
    - **Margem estimada** (em %)
    - **Custo de frete estimado** (R$)
    - **Justificativa detalhada** com os fatores considerados

11.6 Chat sobre o Edital (IA)

Na aba **Chat** (disponível somente após o processamento):

1. Digite sua pergunta no campo de texto.
2. Pressione **Enter** ou clique em **"Enviar"**.
3. A resposta aparecerá com:
   - Texto explicativo gerado pela IA
   - **Fontes:** páginas e trechos do edital utilizados para embasar a resposta
4. O histórico de perguntas é salvo e pode ser consultado novamente.

Exemplos de perguntas:

- *"Quais são os documentos exigidos para habilitação?"*
- *"O edital aceita consórcio de empresas?"*
- *"Qual a multa por atraso na entrega?"*
- *"Existe cota reservada para MEI ou ME?"*

11.7 Gerenciamento de Produtos

Acesse **"Meus Produtos"** no menu lateral.

11.7.1 Cadastrar produto

1. Clique em **"Novo Produto"**.
2. Preencha os campos:
   - **Nome** e **Descrição** do produto
   - **Categoria** (Medicamento, Material Hospitalar, Equipamento, Serviço, Outro)
   - **Custo unitário** (R$) e **Preço de venda** (R$)
   - **Margem mínima exigida** (%) — valor abaixo do qual a licitação não é viável
   - **Peso** (kg/unidade) e **Estado de origem** (UF)
3. Clique em **"Salvar"**.

Importante

O custo, peso e estado de origem são usados diretamente no cálculo de frete e viabilidade. Mantenha esses dados atualizados.

11.7.2 Editar ou excluir produto

Na tabela de produtos, use o ícone de **lápis** para editar ou o ícone de **lixeira** para excluir.

11.8 Notificações

O ícone de **sino** no canto superior direito exibe notificações não lidas:

- **Conclusão de processamento:** edital analisado com sucesso.
- **Erro de processamento:** PDF protegido, ilegível ou falha na IA.

Clique na notificação para ser redirecionado ao edital correspondente.

11.9 Histórico de Análises

Acesse **"Histórico"** no menu lateral para visualizar todas as análises de viabilidade realizadas, com data, produto, edital e classificação.

12 Próximos Passos (Roadmap)

1. **Aprimorar Reranking:** Integrar modelos de Reranking hospedados localmente (como o BAAI/bge-reranker) para eliminar a dependência de APIs de terceiros e reduzir custos de produção.
2. **Pipeline de CI/CD para Validação:** Integrar o script de avaliação RAG (`avaliar_rag.py`) ao pipeline de CI/CD para bloquear deploys se a acurácia semântica média do chat cair abaixo de 80%.
3. **Migração de Storage:** Migrar o armazenamento de PDFs do PostgreSQL para o Google Cloud Storage ou Amazon S3, liberando capacidade do banco de dados relacional à medida que o volume crescer.
4. **Cálculo de frete real:** Integrar com API de geolocalização para calcular a distância porta-a-porta entre o CEP de origem e o CEP de entrega.
5. **Alertas proativos:** Implementar monitoramento periódico do PNCP e notificações automáticas quando novas licitações compatíveis com o produto cadastrado forem publicadas.
6. **Análise multi-produto:** Suporte a análise de viabilidade quando o fornecedor oferece múltiplos itens em uma mesma licitação.

13 Referências

1. BRASIL. Portal Nacional de Contratações Públicas (PNCP). Disponível em: https://www.pncp.gov.br. Acesso em: maio 2026.

2. FASTAPI. FastAPI — documentação oficial. Disponível em: https://fastapi.tiangolo.com. Acesso em: maio 2026.

3. ANGULAR. Angular — documentação oficial. Disponível em: https://angular.io. Acesso em: maio 2026.

4. QDRANT. Qdrant — Vector Search Engine. Disponível em: https://qdrant.tech. Acesso em: maio 2026.

5. GROQ. Groq API documentation. Disponível em: https://console.groq.com/docs. Acesso em: maio 2026.

6. CELERY. Celery — Distributed Task Queue. Disponível em: https://docs.celeryq.dev. Acesso em: maio 2026.

7. LEWIS, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, v. 33, p. 9459–9474, 2020.

8. PYMUPDF. PyMuPDF — documentation. Disponível em: https://pymupdf.readthedocs.io. Acesso em: maio 2026.

9. FASTEMBED. fastembed — documentation. Disponível em: https://qdrant.github.io/fastembed. Acesso em: maio 2026.

10. ANTT. Tabela de Custos Operacionais (TACO) — Portaria SUROC-3/2026. Agência Nacional de Transportes Terrestres, 2026.

11. ES, Sam et al. RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217, 2023.

12. BRASIL. Lei nº 14.133, de 1º de abril de 2021 — Nova Lei de Licitações e Contratos Administrativos. Brasília: Congresso Nacional, 2021.

A Estrutura de Diretórios do Projeto

```
licitiview/
          licitacoes-ws/              # Backend FastAPI
                alembic/                # Migrations de banco de dados
                      versions/
                          001_add_produto_logistica.py
                          002_add_notificacao.py
                rag/
                      edital_rag.py       # Pipeline RAG completo
                routes/
                      auth_routes.py      # Endpoints de autenticação
                      admin_routes.py     # Endpoints administrativos
                      cliente_routes.py   # Endpoints do cliente
                models.py               # Modelos SQLAlchemy
                schemas.py              # Schemas Pydantic
                database.py             # Configuração do banco
                dependencies.py         # Dependências FastAPI (auth)
                main.py                 # Aplicação FastAPI + lifespan
                worker.py               # Configuração Celery
                tasks.py                # Tasks Celery
                Dockerfile
                docker-compose.yml
                requirements.txt

          liciteview-angular/         # Frontend Angular
                src/
                    app/
                        core/
                              models/     # Interfaces TypeScript
                              services/   # Serviços HTTP + lógica
                              interceptors/
                        features/
                              auth/       # Login e cadastro
                              pesquisa/   # Busca de licitações
                              edital/     # Detalhe + RAG + chat
                              produtos/   # CRUD de produtos
                        layout/         # App shell + notificações

          docs/                       # Documentação técnica
              fase-1-auth-busca.md
              fase-2-produtos.md
              fase-3-rag-async.md
              fase-4-chat.md
              fase-5-viabilidade.md
              fase-6-notificacoes-pdf.md
              decisoes-tecnicas.md
              deploy-gcp.md
```

```
                deploy-docker.md
                troubleshooting.md
                documentacao-tecnica.tex
```

Listing 4: Estrutura de diretórios

B Esquema do Banco de Dados

```sql
-- Criacao de Enums (tipados no PostgreSQL)
CREATE TYPE perfil_usuario AS ENUM ('ADMIN', 'FORNECEDOR');
CREATE TYPE categoria_produto AS ENUM ('MEDICAMENTO', 'MATERIAL_HOSPITALAR', 'EQUIPAMENTO', 'SERVICO', 'OUTRO');
CREATE TYPE status_processamento AS ENUM ('PENDENTE', 'PROCESSANDO', 'PROCESSADO', 'ERRO');
CREATE TYPE embedding_status AS ENUM ('PENDENTE', 'GERADO', 'ERRO');
CREATE TYPE classificacao_viabilidade AS ENUM ('VALE_A_PENA', 'ATENCAO', 'NAO_RECOMENDADO');
CREATE TYPE status_pergunta AS ENUM ('PENDENTE', 'RESPONDIDA', 'ERRO');
CREATE TYPE status_execucao AS ENUM ('PENDENTE', 'EXECUTANDO', 'CONCLUIDO', 'ERRO');
CREATE TYPE tipo_processo AS ENUM ('INGESTAO_PDF', 'SEGMENTACAO', 'EMBEDDING', 'RESUMO', 'ANALISE_VIABILIDADE');

-- Tabela de Usuarios
CREATE TABLE usuario (
    id            UUID PRIMARY KEY,
    nome          VARCHAR(255) NOT NULL,
    email         VARCHAR(255) UNIQUE NOT NULL,
    senha_hash    TEXT NOT NULL,
    razao_social  VARCHAR(255) NOT NULL,
    cnpj          VARCHAR(18) UNIQUE NOT NULL,
    cep           VARCHAR(9),
    perfil        perfil_usuario DEFAULT 'FORNECEDOR',
    ativo         BOOLEAN NOT NULL DEFAULT TRUE,
    criado_em     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    atualizado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabela de Produtos
CREATE TABLE produto (
    id                UUID PRIMARY KEY,
    usuario_id        UUID NOT NULL REFERENCES usuario(id) ON DELETE CASCADE,
    nome              VARCHAR(255) NOT NULL,
    descricao         TEXT,
    categoria         categoria_produto NOT NULL DEFAULT 'OUTRO',
```

```
    ncm                VARCHAR(10),
    preco_venda        NUMERIC(15,4),
    custo_produto      NUMERIC(15,4) NOT NULL,
    margem_minima      NUMERIC(5,4) NOT NULL,
    peso_kg            NUMERIC(10,4),
    impostos           NUMERIC(5,2) NOT NULL DEFAULT 0,
    cep                VARCHAR(9),
    uf                 VARCHAR(2),
    capacidade_mensal  INTEGER,
    ativo              BOOLEAN NOT NULL DEFAULT TRUE,
    criado_em          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    atualizado_em      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabela de Editais
CREATE TABLE edital (
    id                    UUID PRIMARY KEY,
    id_pncp               VARCHAR(100) UNIQUE NOT NULL,
    titulo                VARCHAR(500),
    orgao                 VARCHAR(500) NOT NULL,
    numero                VARCHAR(100),
    modalidade            VARCHAR(100),
    objeto                TEXT NOT NULL,
    valor_estimado        NUMERIC(18,4),
    data_publicacao       DATE,
    data_abertura         DATE,
    url_origem            VARCHAR(1000),
    url_pdf               VARCHAR(1000),
    pdf_conteudo          BYTEA,
    hash_documento        VARCHAR(64),
    status_processamento  status_processamento NOT NULL DEFAULT 'PENDENTE',
    criado_em             TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    atualizado_em         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabela de Secoes de Editais
CREATE TABLE secao_edital (
    id             UUID PRIMARY KEY,
    edital_id      UUID NOT NULL REFERENCES edital(id) ON DELETE CASCADE,
    titulo         VARCHAR(500),
    ordem          INTEGER NOT NULL,
    texto_bruto    TEXT NOT NULL,
    chunk_tokens   INTEGER,
    pagina_inicial INTEGER,
    pagina_final   INTEGER
);

```

```sql
-- Tabela de Chunks de Editais
CREATE TABLE chunk_edital (
    id               UUID PRIMARY KEY,
    edital_id        UUID NOT NULL REFERENCES edital(id) ON DELETE CASCADE,
    secao_id         UUID REFERENCES secao_edital(id) ON DELETE SET NULL,
    ordem            INTEGER NOT NULL,
    texto            TEXT NOT NULL,
    tokens           INTEGER NOT NULL,
    pagina_inicial   INTEGER,
    pagina_final     INTEGER,
    hash_chunk       VARCHAR(64),
    embedding_status embedding_status NOT NULL DEFAULT 'PENDENTE',
    qdrant_point_id  VARCHAR(100),
    embedding_model  VARCHAR(100),
    criado_em        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabela de Resumos de Editais
CREATE TABLE resumo_edital (
    id            UUID PRIMARY KEY,
    edital_id     UUID NOT NULL REFERENCES edital(id) ON DELETE CASCADE,
    versao        INTEGER NOT NULL DEFAULT 1,
    json_resumo   JSONB NOT NULL,
    modelo_usado  VARCHAR(100),
    versao_prompt VARCHAR(50),
    ativo         BOOLEAN NOT NULL DEFAULT TRUE,
    gerado_em     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabela de Variaveis Extraidas de Editais
CREATE TABLE variavel_viabilidade (
    id             UUID PRIMARY KEY,
    edital_id      UUID NOT NULL REFERENCES edital(id) ON DELETE CASCADE,
    nome_variavel  VARCHAR(100) NOT NULL,
    valor_extraido TEXT NOT NULL,
    unidade        VARCHAR(50),
    confianca      NUMERIC(4,3),
    modelo_usado   VARCHAR(100),
    gerado_em      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabela de Tarifas Rodoviarias de Referencia ANTT
CREATE TABLE tabela_frete_antt (
    id              UUID PRIMARY KEY,
    tabela_origem   VARCHAR(1) NOT NULL,
    tipo_carga_cod  INTEGER NOT NULL,
    tipo_carga_nome VARCHAR(100) NOT NULL,
```

```
    coeficiente     VARCHAR(20) NOT NULL,
    unidade         VARCHAR(10) NOT NULL,
    eixos_2         NUMERIC(10,4),
    eixos_3         NUMERIC(10,4),
    eixos_4         NUMERIC(10,4),
    eixos_5         NUMERIC(10,4),
    eixos_6         NUMERIC(10,4),
    eixos_7         NUMERIC(10,4),
    eixos_9         NUMERIC(10,4),
    versao_portaria VARCHAR(50) NOT NULL DEFAULT 'SUROC-3-2026',
    data_vigencia   DATE NOT NULL DEFAULT '2026-03-13',
    ativo           BOOLEAN NOT NULL DEFAULT TRUE
);

-- Tabela de Analises de Viabilidade Realizadas
CREATE TABLE analise_viabilidade (
    id                      UUID PRIMARY KEY,
    edital_id               UUID NOT NULL REFERENCES edital(id) ON DELETE
        CASCADE,
    produto_id              UUID NOT NULL REFERENCES produto(id) ON DELETE
        CASCADE,
    usuario_id              UUID NOT NULL REFERENCES usuario(id) ON DELETE
        CASCADE,
    margem_minima_snapshot NUMERIC(5,4) NOT NULL,
    preco_venda_snapshot   NUMERIC(15,4) NOT NULL,
    custo_produto_snapshot NUMERIC(15,4) NOT NULL,
    distancia_km           NUMERIC(10,2),
    custo_frete_estimado   NUMERIC(15,4),
    custo_total_estimado   NUMERIC(15,4),
    receita_estimada       NUMERIC(15,4),
    margem_estimada        NUMERIC(5,4),
    dentro_margem          BOOLEAN NOT NULL,
    classificacao          classificacao_viabilidade NOT NULL,
    justificativa          TEXT,
    modelo_usado           VARCHAR(100),
    versao_prompt          VARCHAR(50),
    gerado_em              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabela de Perguntas Feitas nos Editais (Chat RAG)
CREATE TABLE pergunta_edital (
    id            UUID PRIMARY KEY,
    edital_id     UUID NOT NULL REFERENCES edital(id) ON DELETE CASCADE,
    usuario_id    UUID NOT NULL REFERENCES usuario(id) ON DELETE CASCADE,
    pergunta      TEXT NOT NULL,
    status        status_pergunta NOT NULL DEFAULT 'PENDENTE',
    criada_em     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
    resposta      TEXT,
    confianca     NUMERIC(4,3),
    trecho_fonte  TEXT,
    pagina_fonte  INTEGER,
    modelo_usado  VARCHAR(100),
    respondida_em TIMESTAMPTZ
);

-- Tabela de Notificacoes enviadas ao Usuario
CREATE TABLE notificacao (
    id         UUID PRIMARY KEY,
    usuario_id UUID NOT NULL REFERENCES usuario(id) ON DELETE CASCADE,
    tipo       VARCHAR(50) NOT NULL,
    titulo     VARCHAR(500) NOT NULL,
    mensagem   TEXT,
    edital_id  UUID REFERENCES edital(id) ON DELETE SET NULL,
    referencia VARCHAR(200),
    lida       BOOLEAN NOT NULL DEFAULT FALSE,
    criada_em  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tabela de Logs de Execucao de Tarefas Assincronas (Celery)
CREATE TABLE execucao_processo (
    id                UUID PRIMARY KEY,
    edital_id         UUID REFERENCES edital(id) ON DELETE SET NULL,
    pergunta_id       UUID REFERENCES pergunta_edital(id) ON DELETE SET
        NULL,
    tipo_processo     tipo_processo NOT NULL,
    status            status_execucao NOT NULL DEFAULT 'PENDENTE',
    tentativas        INTEGER NOT NULL DEFAULT 0,
    tempo_execucao_ms BIGINT,
    worker_id         VARCHAR(200),
    erro_mensagem     TEXT,
    iniciado_em       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    finalizado_em     TIMESTAMPTZ
);
```

Listing 5: DDL completo e sincronizado com os modelos SQLAlchemy